

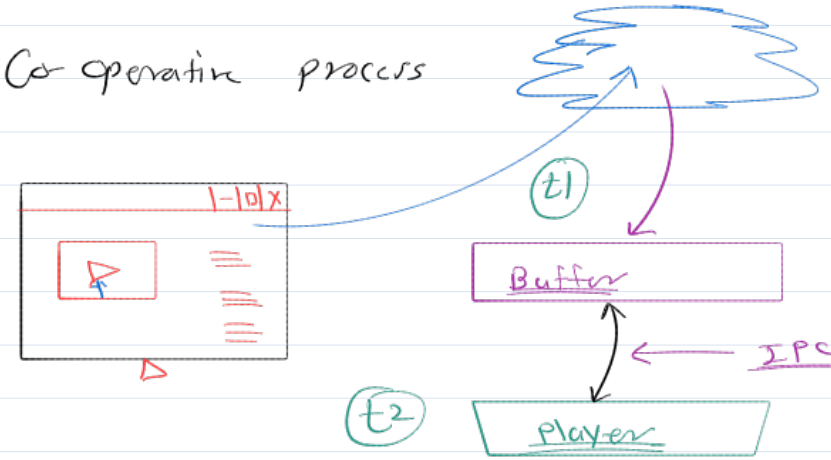
Concepts of Operating Systems And Linux Operating Systems

Trainer: Kiran Jaybhave

① Independent process

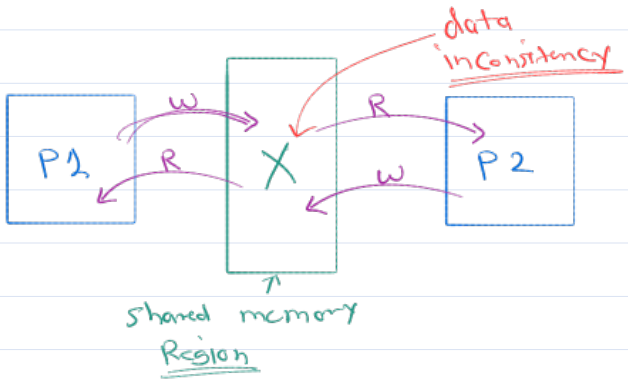


② Co-operating process



① message passing model
- Pipe
- msg Queue
- signal
- Socket (X)

② shared memory.

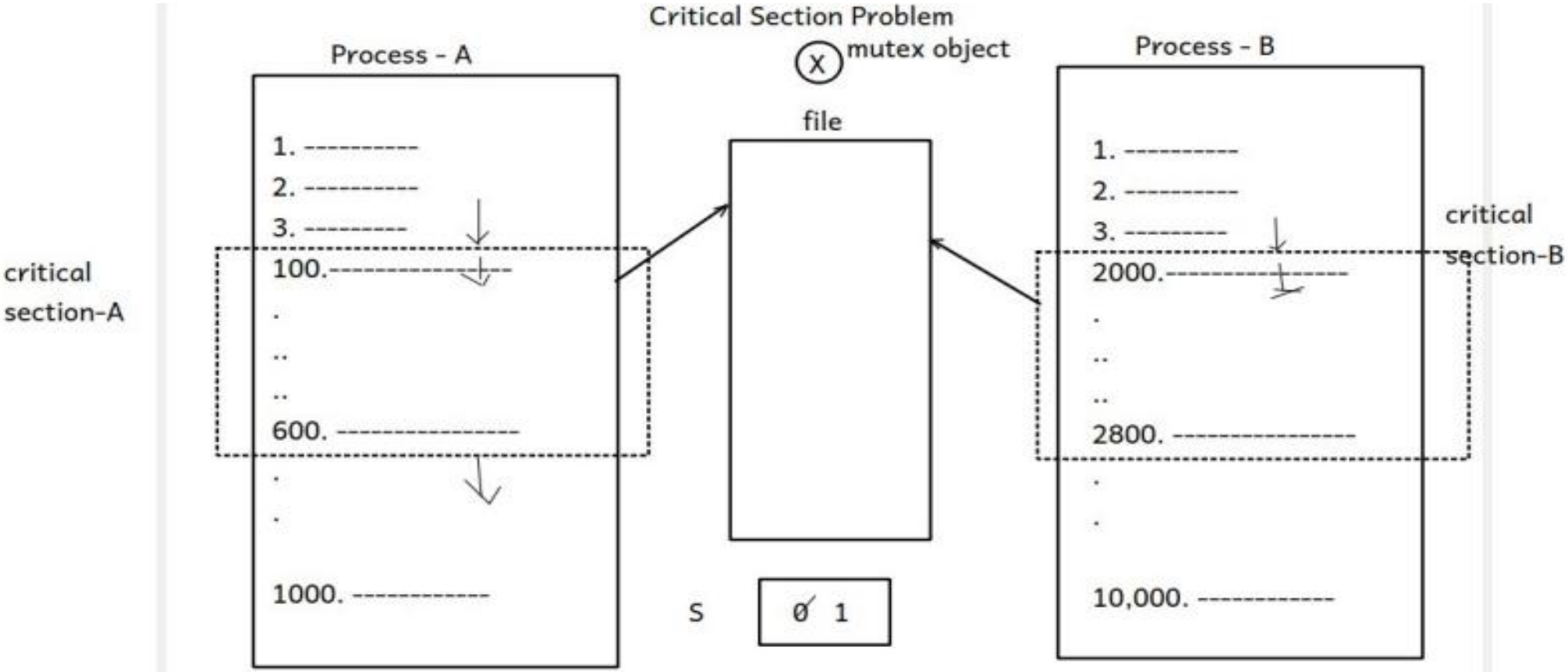


Process Coordination / Process Synchronization

Why Process Co-ordination/Synchronization?

- If concurrently executing co-operative processes are accessing common resources, then conflictions may takes place, which may results into the problem of **data inconsistency**, and hence to avoid this problem coordination / synchronization between these processes is required.
- Multiple processes accessing same resource at the same time, is known as **"race condition".** When race condition occurs, resource may get corrupted (unexpected results).
- **Peterson's problem**, if two processes are trying to modify same variable at the same time, it can produce unexpected results.
- Code block to be executed by only one process at a time is referred as **Critical section**.
- If multiple processes execute the same code concurrently it may produce undesired results.
- To resolve race condition problem, one process can access resource at a time. This can be done using **sync** objects/primitives given by OS.

SUNBEAM



"data incosistency" problem occure in above case only when both the sections of process A & B are runniing at a same time, and hence these sections are reffered as critical section, and hence data inconsistency problem may occure when two or more processes are running in their critical sections at a same time, and this problem is also reffered as "critical section problem".

➢Synchronization Tools:

1. **Semaphore:**
2. **Mutex :**

➢Semaphore:

- Semaphore was suggested by Dijkstra scientist (dutch math)
- Semaphore is a counter
- Semaphore is a sync primitive given by OS.

❑On semaphore two operations are supported:

➢wait operation: decrement op: P operation:

1. Semaphore count is decremented by 1.
2. If cnt < 0, then calling process is blocked(block the current process).
3. Typically wait operation is performed before accessing the resource.

SUNBEAM

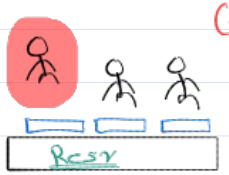
* Counting

① Wait / decrement / P oprn
   ① Before accessing Rsr
   ② Decrement Semaphor Cnt
   ③ Cnt < 0 ⇒ True ⇒ Block process

② Signal / Increment / V oprn
   ① After Relasing Rsr
   ② Increment semaphor cnt
   ③ Wake up any block process.

(↑) V oprn

(↓) P oprn



Cnt
3

2 < 0 ⇒ F
1 < 0 ⇒ F
0 < 0 ⇒ F
-1 < 0 ⇒ T
-2 < 0 ⇒ T
-3 < 0 ⇒ T

Block
Block

-2

---

* Binary Semaphor.

V oprn
P oprn



Cnt
1
0
1
0

➢**signal operation: increment op: V operation:**

1. semaphore count is incremented by 1.
2. if one or more processes are blocked on the semaphore, then wake up one of the process.
3. typically signal operation is performed after releasing the resource.

Q. If sema count = -n, how many processes are waiting on that semaphore?

Answer: "n" processes waiting

• There are two types of semaphore

i. **Binary semaphore** **:** can be used when at a time resource can be acquired by only one process.

- Allows only 1 process to access resource at a time or used as a flag/condition.

ii. **Counting / Classic semaphore** **:** can be used when at a time resource can be acquired by more than one processes

SUNBEAM

2. Mutex Object:

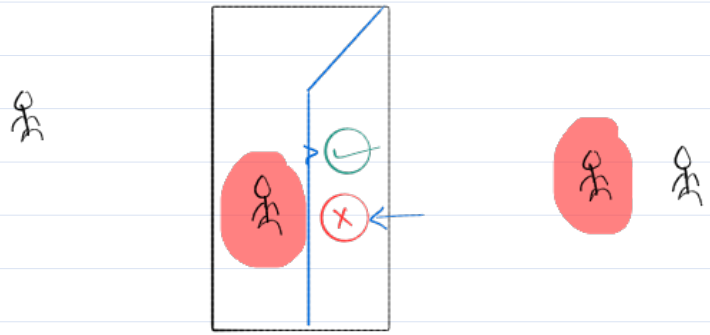
- Mutex is used to ensure that only one process can access the resource at a time.
- Functionally it is same as "binary semaphore".
- Mutex can be unlocked by the same process/thread, which had locked it.

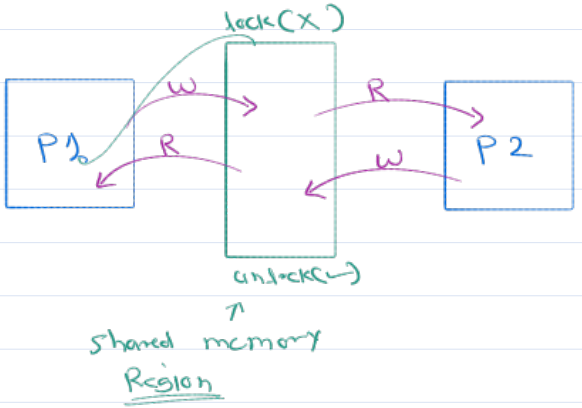
➢ Semaphore vs Mutex

- ✓S: Semaphore can be decremented by one process and incremented by same or another process.
- ✓M: The process locking the mutex is owner of it. Only owner can unlock that mutex.
- ✓S: Semaphore can be counting or binary.
- ✓M: Mutex is like binary semaphore. Only two states: locked and unlocked.
- ✓S: Semaphore can be used for counting, mutual exclusion or as a flag.
- ✓M: Mutex can be used only for mutual exclusion.

SUNBEAM

* Mutex (lock & unlock)

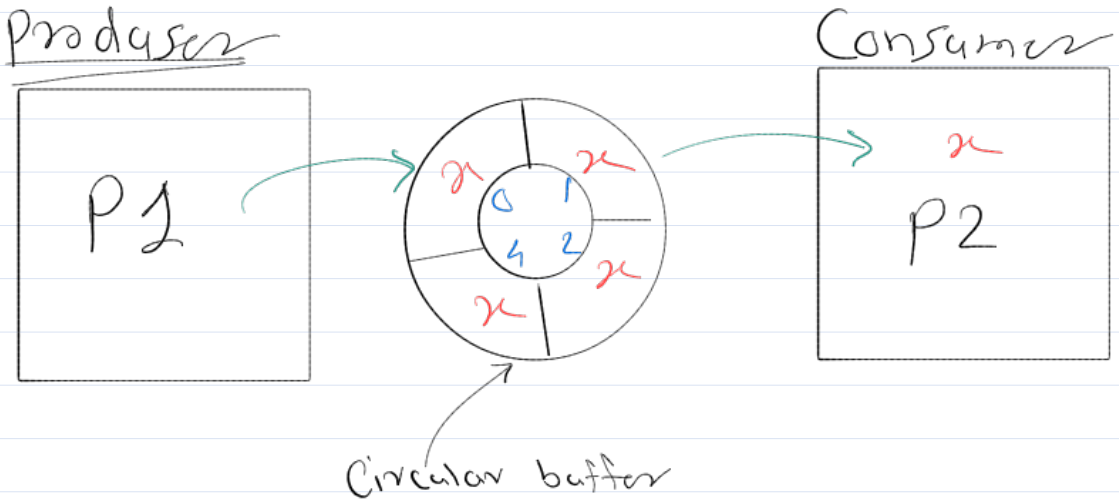




Producer-Consumer Problem

- A classic multi-process synchronization problem where two processes share a common, fixed-size buffer used as a queue.
- **The Processes:**
  - **Producer:** Generates data and puts it into the buffer.
  - **Consumer:** Removes data from the buffer and consumes it.
- **Core Challenges:**
  - Prevent the **producer** from adding data when the buffer is **full**.
  - Prevent the **consumer** from removing data when the buffer is **empty**.
  - Prevent both from **accessing the buffer at the same time** to avoid race conditions.
- **The Solution Mechanism:**
  - If the buffer is full, the **producer** must **sleep or discard data**. It is later **woken up** by the consumer.
  - If the buffer is empty, the **consumer** must **sleep**. It is later **woken up** by the producer.
  - This coordination ensures efficient and safe use of the shared buffer.

SUNBEAM


Producer
P1
Consumer
P2
0
1
2
4
Circular buffer

➢Deadlock:

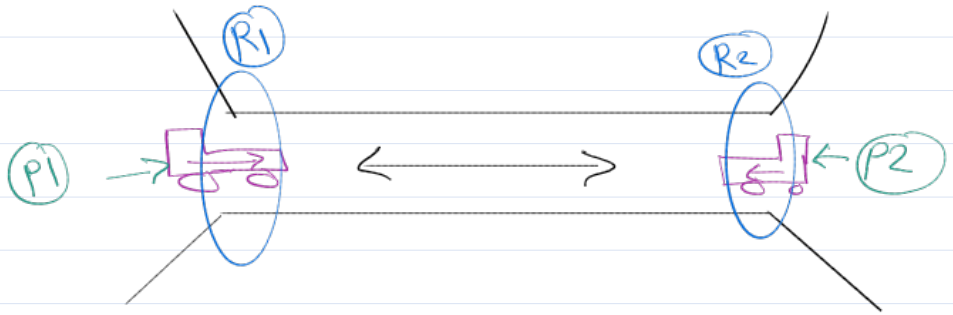
- There are four necessary and sufficient conditions to occur deadlock / characteristics of deadlock:

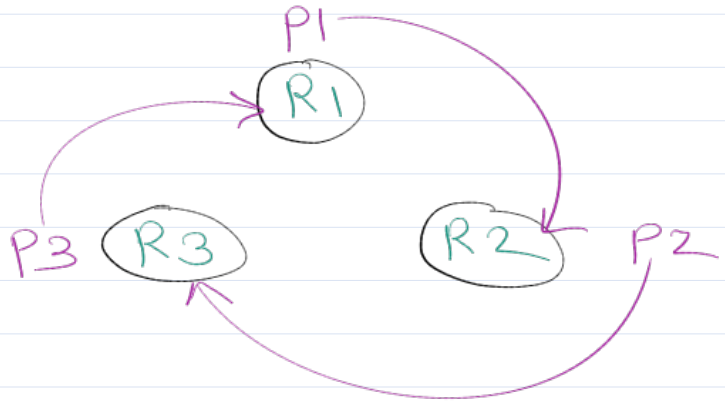
1. **Mutual Exclusion:** at a time resource can be acquired by only one process.
2. **No Preemption:** control of the resource cannot be taken away forcefully from any process.
3. **Hold & Wait:** every process is holding one resource and waiting for the resource which is held by another process.
4. **Circular Wait:** if process P1 is holding resource and waiting for the resource held by another process P2, and process P2 is also holding one resource and waiting for the resource held by process P1.

SUNBEAM

1 Deadlock

1. mutual enclusion
2. Hold & wait
3. No-Pxemption
4. Circular wait





The deadlock handling methods are there:

1. **Deadlock Prevention:**
   - OS Syscalls are designed so that at least one deadlock condition does not hold true.
   - In UNIX multiple semaphore operations can be done at the same time.

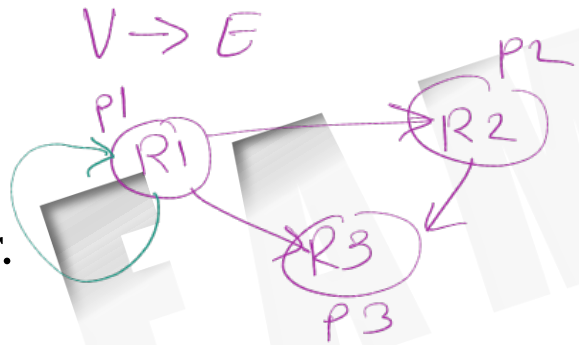
2. **Deadlock Detection & Avoidance:**
   - before allocating resources for processes all input can be given to deadlock detection algorithm in advanced and if there are chances to occur deadlock then it can be avoided by doing necessary changes.

❖**There are two deadlock detection & avoidance algorithms:**

1. **Resource Allocation Graph Algorithm**
2. **Banker's Algorithm**
3. **Safe state algorithm**

SUNBEAM

- Processes declare the required resources in advanced, based on which OS decides whether resource should be given to the process or not.
- **Algorithms used for this are:**
  - **Resource allocation graph:**
    - OS maintains graph of resources and processes.
    - A cycle in graph indicate circular wait will occur.
    - In this case OS can deny a resource to a process.



  - **Banker's algorithm:**
    - A bank always manage its cash so that they can satisfy all customers.
  - **Safe state algorithm:**
    - OS maintains statistics of number of resources and number processes.
    - Based on stats it decides whether giving resource to a process is safe or not (using a formula):
      - **Max num of resources required < Num of resources + Num of processes**
        - If condition is true, deadlock will never occur.
        - If condition is false, deadlock may occur

SUNBEAM

❑Starvation:

- The process not getting enough CPU time for its execution.
- Process is in ready state/queue.
- Reason: Lower priority (CPU is busy in executing high priority process).

❑Deadlock:

- The process not getting the resource for its execution.
- Process is in waiting state/queue indefinitely.
- Reason: Resource is blocked by another process (and there is circular wait).

SUNBEAM



Thank you!

Kiran Jaybhave

email – kiran.jaybhave@sunbeaminfo.com